



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2628001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

25ADC01 - ARTIFICIAL INTELLIGENCE AND APPLICATIONS

NAME :	K YUKESH
REG NUMBER :	25cs304
CLASS & SECTION :	CSE-D
SUBJECT CODE :	25ADC01
SUBJECT NAME :	ARTIFICIAL INTELLIGENCE AND APPLICATIONS
ACTIVITY NO :	1

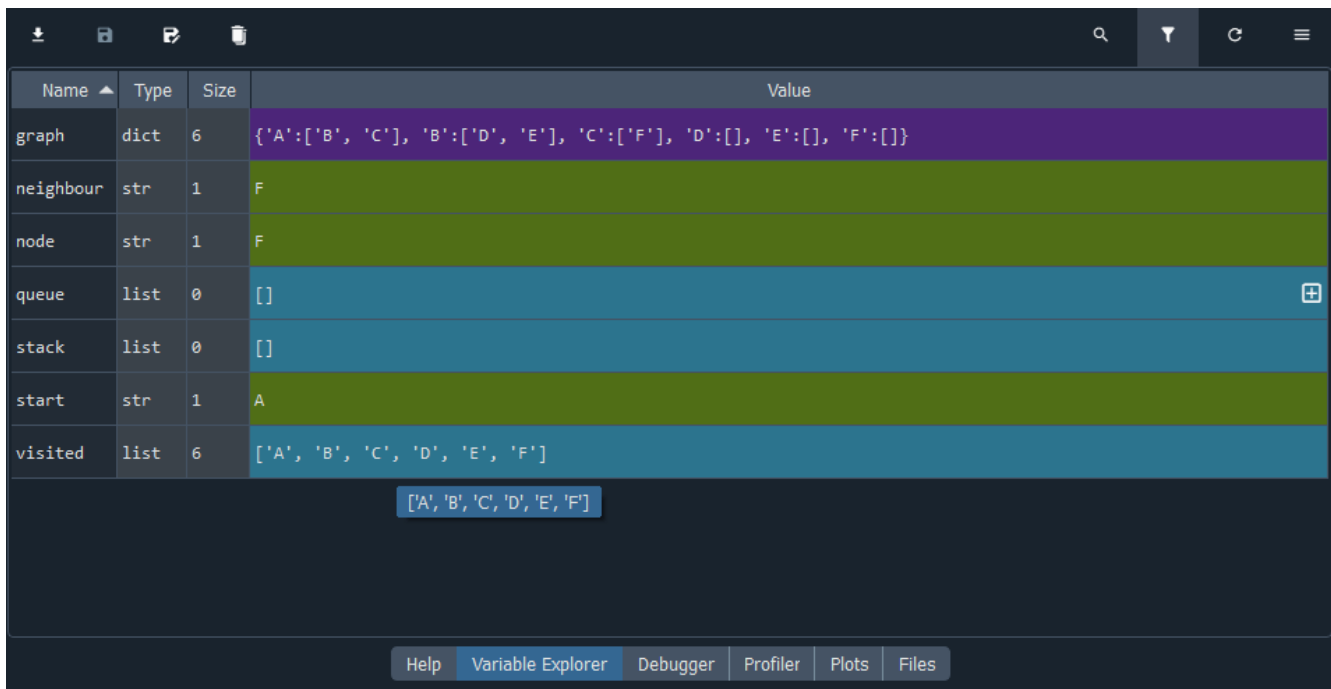
EXP NO : 1A	Breadth First Search	DATE :
AIM: To implement the Breadth First Search (BFS) algorithm for graph traversal using Python. OBJECTIVE: To understand graph traversal using the BFS technique and Queue data structure. SOFTWARE REQUIREMENTS: • Python 3.x • Spyder IDE THEORY: Breadth First Search (BFS) is a graph traversal algorithm that visits all nodes level by level. It starts from a source node and explores all neighboring nodes before moving to the next level. BFS uses a Queue data structure which follows the FIFO (First In First Out) principle. ALGORITHM: Step 1: Create a graph using an adjacency list. Step 2: Initialize an empty list called visited. Step 3: Initialize an empty queue. Step 4: Add the starting node to visited and queue. Step 5: Remove a node from the front of the queue. Step 6: Visit the node and print it. Step 7: Add all unvisited neighboring nodes to the queue. Step 8: Repeat Steps 5 to 7 until the queue becomes empty. Step 9: Stop. PROGRAM: <pre>graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': [], 'F': [] } visited = [] queue = [] start = 'A'</pre>		

```
visited.append(start)
queue.append(start)
print("BFS Traversal:")
while queue:
    node = queue.pop(0)

    print(node, end=" ")

    for neighbour in graph[node]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
print("\nVisited Nodes:", visited)
```

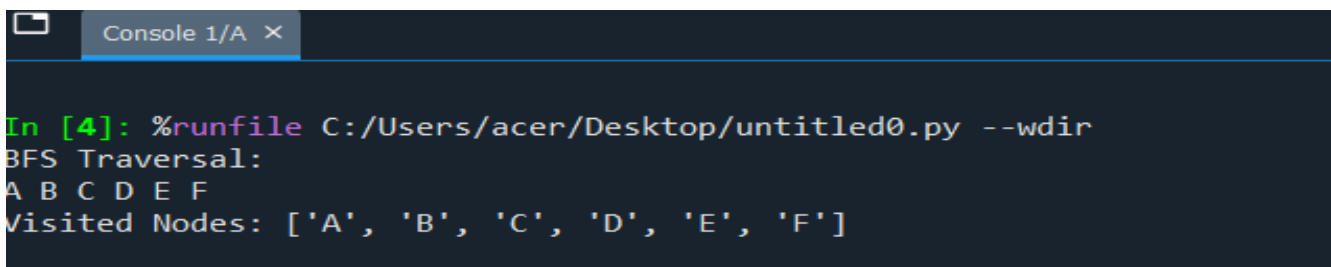
OUTPUT :



Name ▲	Type	Size	Value
graph	dict	6	{'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': [], 'F': []}
neighbour	str	1	F
node	str	1	F
queue	list	0	[]
stack	list	0	[]
start	str	1	A
visited	list	6	['A', 'B', 'C', 'D', 'E', 'F']

['A', 'B', 'C', 'D', 'E', 'F']

Help Variable Explorer Debugger Profiler Plots Files



```
In [4]: %runfile C:/Users/acer/Desktop/untitled0.py --wdir
BFS Traversal:
A B C D E F
Visited Nodes: ['A', 'B', 'C', 'D', 'E', 'F']
```

RESULT:

Thus, the Breadth First Search (BFS) algorithm was successfully implemented and executed using Python.

EXP NO : 1B	Depth First Search	DATE :
AIM: To implement the Depth First Search (DFS) algorithm for graph traversal using Python. OBJECTIVE: To understand graph traversal using the DFS technique and Stack/Recursion. SOFTWARE REQUIREMENTS: • Python 3.x • Spyder IDE THEORY: Depth First Search (DFS) is a graph traversal algorithm that explores a branch completely before moving to another branch. It uses recursion or a stack for traversal. DFS follows the principle of going deeper into the graph before backtracking. ALGORITHM: Step 1: Create a graph using an adjacency list. Step 2: Initialize an empty list called visited. Step 3: Start from the source node. Step 4: Mark the node as visited. Step 5: Print the current node. Step 6: Visit each unvisited neighboring node recursively. Step 7: Continue until all nodes are visited. Step 8: Stop. PROGRAM : <pre># DFS using Stack graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': [] } visited = [] stack = ['A']</pre>		

while stack:

```
node = stack.pop()
```

```
if node not in visited:
```

```
    visited.append(node)
```

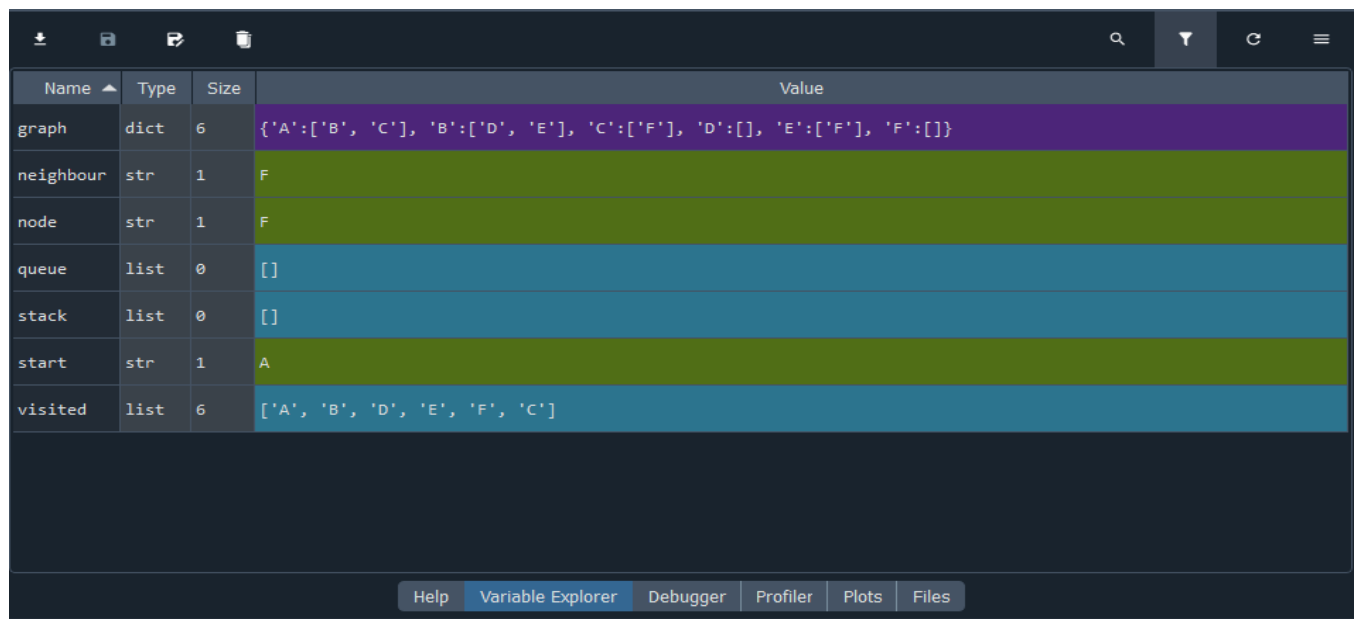
```
    print(node, end=" ")
```

```
    # Reverse order for correct traversal
```

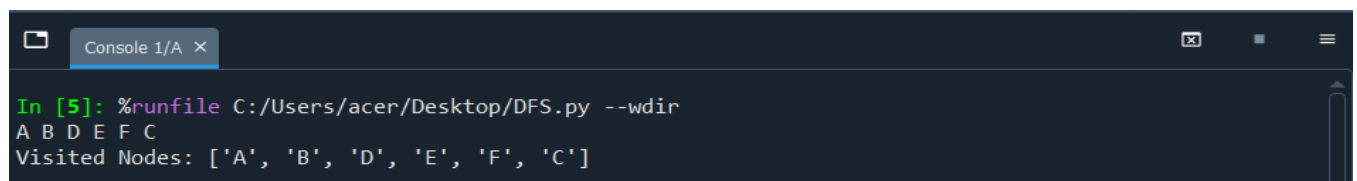
```
    for neighbour in reversed(graph[node]):  
        stack.append(neighbour)
```

```
print("\nVisited Nodes:", visited)
```

OUTPUT :



Name	Type	Size	Value
graph	dict	6	{'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': []}
neighbour	str	1	F
node	str	1	F
queue	list	0	[]
stack	list	0	[]
start	str	1	A
visited	list	6	['A', 'B', 'D', 'E', 'F', 'C']



```
In [5]: %runfile C:/Users/acer/Desktop/DFS.py --wdir  
A B D E F C  
Visited Nodes: ['A', 'B', 'D', 'E', 'F', 'C']
```

RESULT :

Thus, the Depth First Search (DFS) algorithm was successfully implemented and executed using Python

